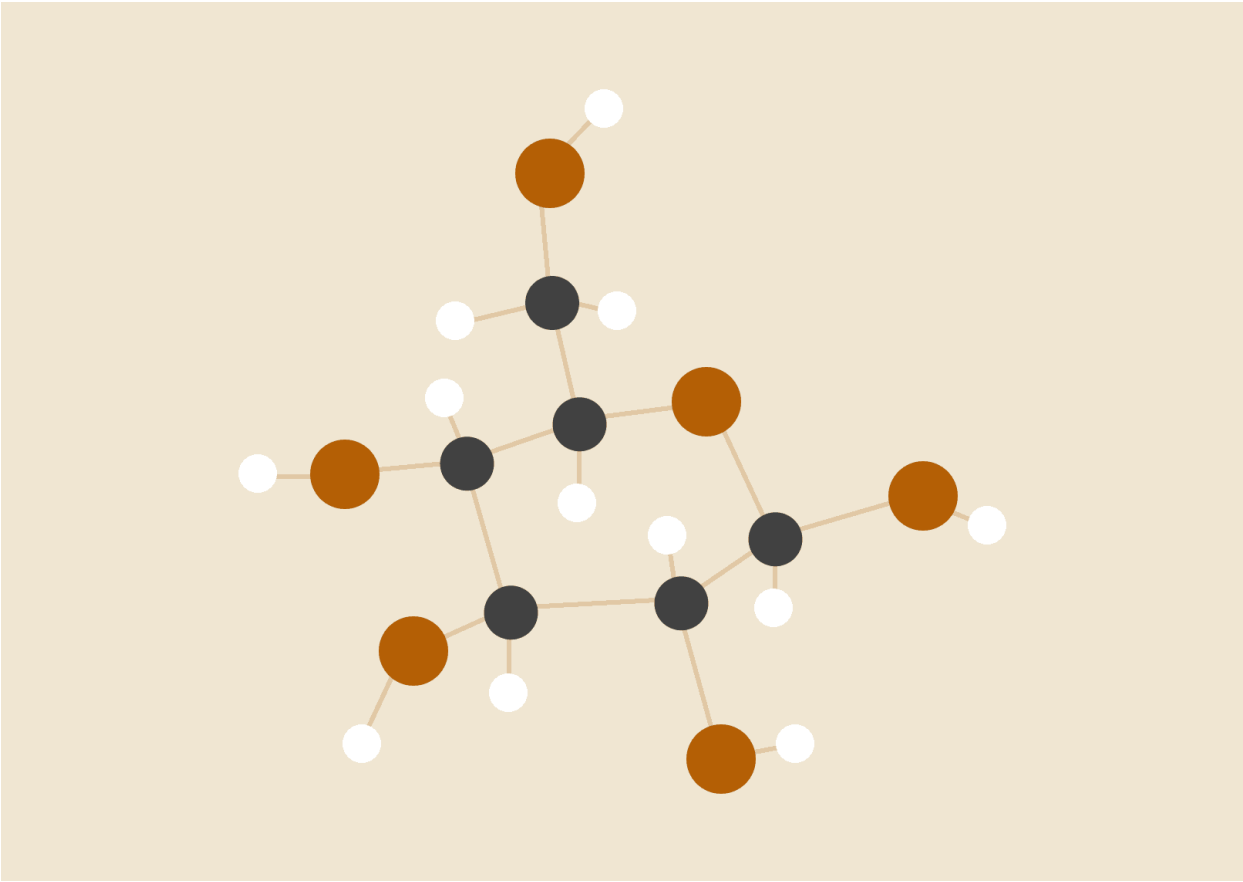
GoScript IDE Manual Tecnico

Proyecto 2



Fernando Andhre Gonzalez Espinoza

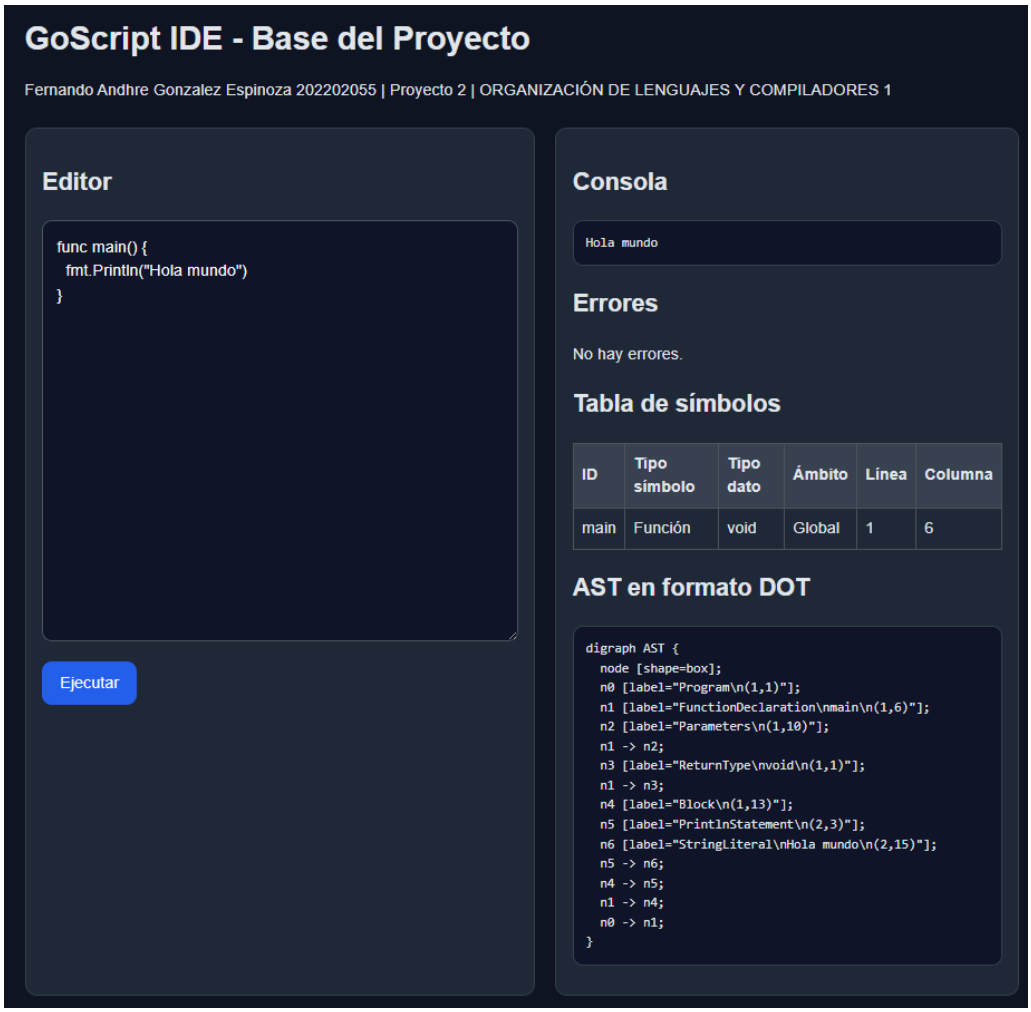
202202055

ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 1

1. Introducción

El presente Manual Técnico describe la estructura interna, funcionamiento, arquitectura y tecnologías utilizadas en el desarrollo del sistema GoScript IDE – Base del Proyecto. El sistema fue diseñado para permitir la escritura, análisis y ejecución controlada de código fuente basado en un lenguaje similar a Go, integrando componentes de análisis léxico, sintáctico y semántico, así como generación de AST, tabla de símbolos, reporte de errores y salida en consola.

Este documento está orientado a fines académicos y de mantenimiento, por lo que explica cómo está construido el sistema, cómo se relacionan sus módulos y cuál es el flujo de procesamiento del código fuente desde que el usuario lo escribe hasta que se muestran los resultados en pantalla.



2. Objetivos técnicos

2.1 Objetivo general

Implementar un entorno de análisis e interpretación para programas escritos en GoScript, permitiendo procesar el código fuente mediante un analizador léxico y sintáctico basado en Jison, construir un Árbol de Sintaxis Abstracta y ejecutar validaciones semánticas para mostrar resultados estructurados al usuario.

2.2 Objetivos específicos

Implementar un frontend que permita ingresar código fuente y visualizar resultados. Desarrollar un backend encargado de procesar el código, ejecutar el parser y generar la respuesta.

- Construir una gramática funcional en Jison para soportar las estructuras principales del lenguaje.
- Generar un AST que represente la estructura del programa.
- Desarrollar un intérprete semántico capaz de ejecutar instrucciones válidas del lenguaje.
- Registrar símbolos encontrados durante la ejecución.
- Detectar y reportar errores léxicos, sintácticos y semánticos.
- Mostrar resultados en consola, tabla de errores, tabla de símbolos y AST en formato DOT.

3. Alcance del sistema

El sistema está diseñado para procesar programas escritos en GoScript dentro del contexto del proyecto académico. Su alcance incluye el análisis e interpretación de múltiples construcciones del lenguaje, tales como variables, tipos primitivos, expresiones aritméticas y lógicas, estructuras de control, funciones, arreglos, slices, matrices, structs y funciones embebidas.

El sistema no está pensado como un compilador completo de producción, sino como una herramienta académica orientada al análisis, validación e interpretación de programas definidos bajo la gramática del proyecto.

4. Tecnologías utilizadas

Para el desarrollo del sistema se utilizaron las siguientes tecnologías:

4.1 Frontend

El frontend fue desarrollado para proporcionar una interfaz visual en la que el usuario puede escribir código, ejecutar el análisis y revisar los resultados obtenidos.

Tecnologías utilizadas:

- React
- TypeScript o JavaScript, según la configuración del frontend
- HTML
- CSS

4.2 Backend

El backend es responsable de recibir el código fuente, ejecutar el parser, interpretar el AST y devolver la respuesta estructurada al frontend.

Tecnologías utilizadas:

- Node.js
- Express
- TypeScript

4.3 Generación de parser

La construcción del analizador léxico y sintáctico se realizó mediante:

- Jison

4.4 Otras utilidades

- Formato DOT para representar el AST
- JSON para el intercambio de datos entre frontend y backend

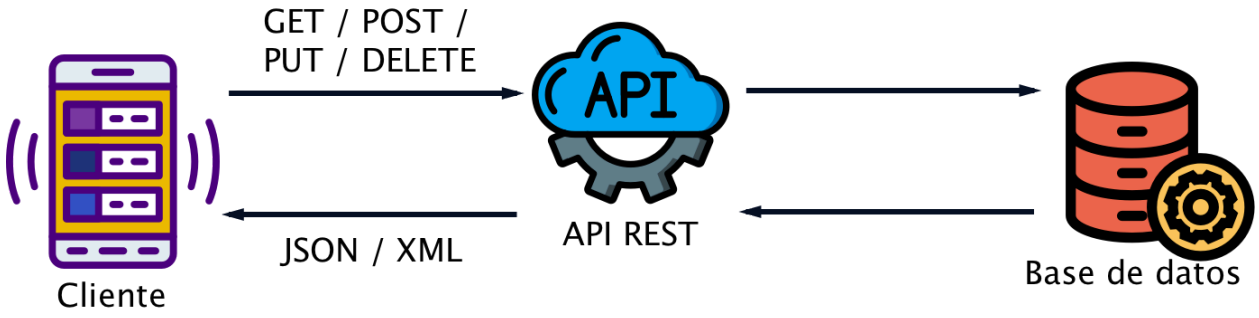
5. Arquitectura general del sistema

La arquitectura del sistema sigue un modelo cliente-servidor.

El usuario interactúa con el frontend, donde escribe el código fuente en el editor. Al presionar el botón de ejecución, el frontend envía el contenido al backend por medio de una solicitud HTTP. El backend recibe el código, ejecuta el proceso de análisis, construye el AST, realiza validaciones semánticas e interpreta el programa. Finalmente, la respuesta generada se envía al frontend, donde se muestra en las diferentes secciones de la interfaz.

Flujo general

Usuario → Frontend → API Backend → Parser/AST/Intérprete → Respuesta JSON → Frontend



6. Estructura del proyecto

El proyecto se encuentra organizado en dos partes principales:

6.1 Frontend

Contiene la interfaz visual y los componentes encargados de:

- capturar el código
- enviar solicitudes al backend
- mostrar la consola
- mostrar errores
- mostrar la tabla de símbolos
- mostrar el AST

6.2 Backend

Contiene la lógica de análisis e interpretación. Dentro de esta parte se encuentran archivos y carpetas relevantes como:

- **src/grammar/**

Contiene la gramática definida en Jison y los archivos generados del parser.

- **src/interpreter/**

Contiene la lógica del intérprete semántico, evaluación de expresiones, manejo de scopes, ejecución de instrucciones y construcción de resultados.

- **src/api/**

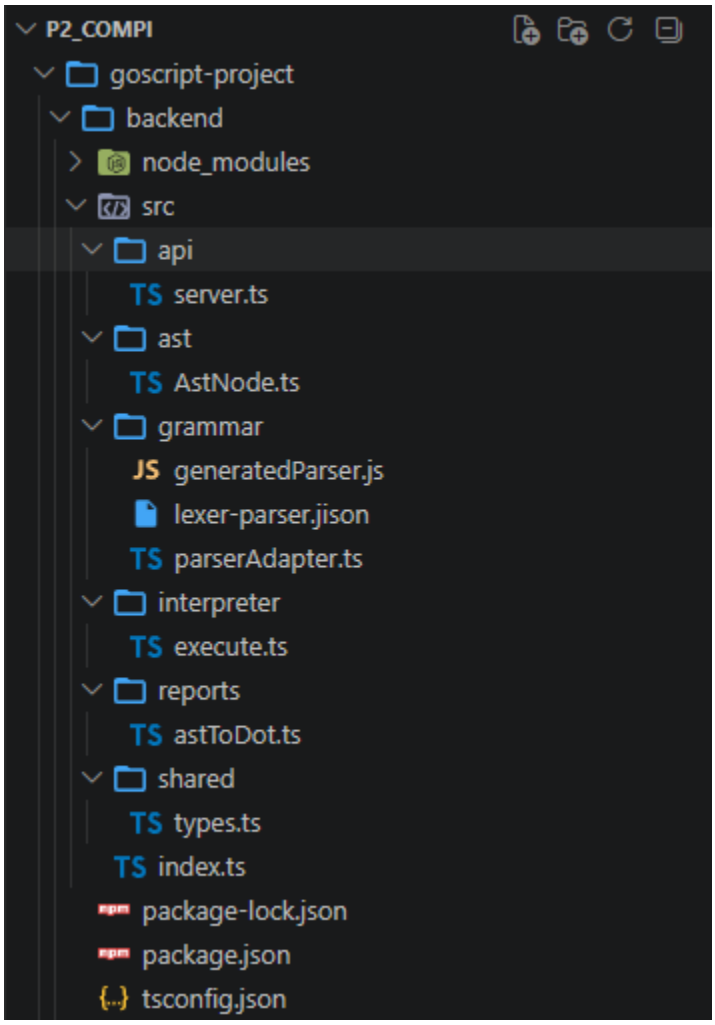
Contiene la parte del servidor que expone la ruta para ejecutar el análisis del código.

- **src/reports/**

Contiene utilidades para generación de salidas estructuradas, como el AST en DOT.

- **src/shared/**

Contiene tipos compartidos utilizados entre distintas partes del sistema.



7. Descripción de módulos

7.1 Módulo de interfaz

Este módulo permite al usuario interactuar con el sistema. Incluye:

- editor de código
- botón de ejecución
- consola
- tabla de errores
- tabla de símbolos
- área de AST

7.2 Módulo de servidor

Este módulo recibe el código fuente enviado por el frontend y coordina el análisis del programa.

Sus responsabilidades son:

- recibir la solicitud HTTP
- validar el cuerpo de la petición
- llamar al parser y al intérprete
- devolver resultados al frontend

7.3 Módulo de gramática

Implementa el análisis léxico y sintáctico por medio de Jison.

Sus responsabilidades incluyen:

- reconocimiento de tokens
- validación estructural del código
- construcción de nodos del AST

7.4 Módulo de interpretación

Se encarga de:

- recorrer el AST
- manejar variables y scopes
- validar tipos
- ejecutar instrucciones
- controlar flujo
- registrar errores
- construir tabla de símbolos

7.5 Módulo de reportes

Genera estructuras de salida legibles para el usuario, como:

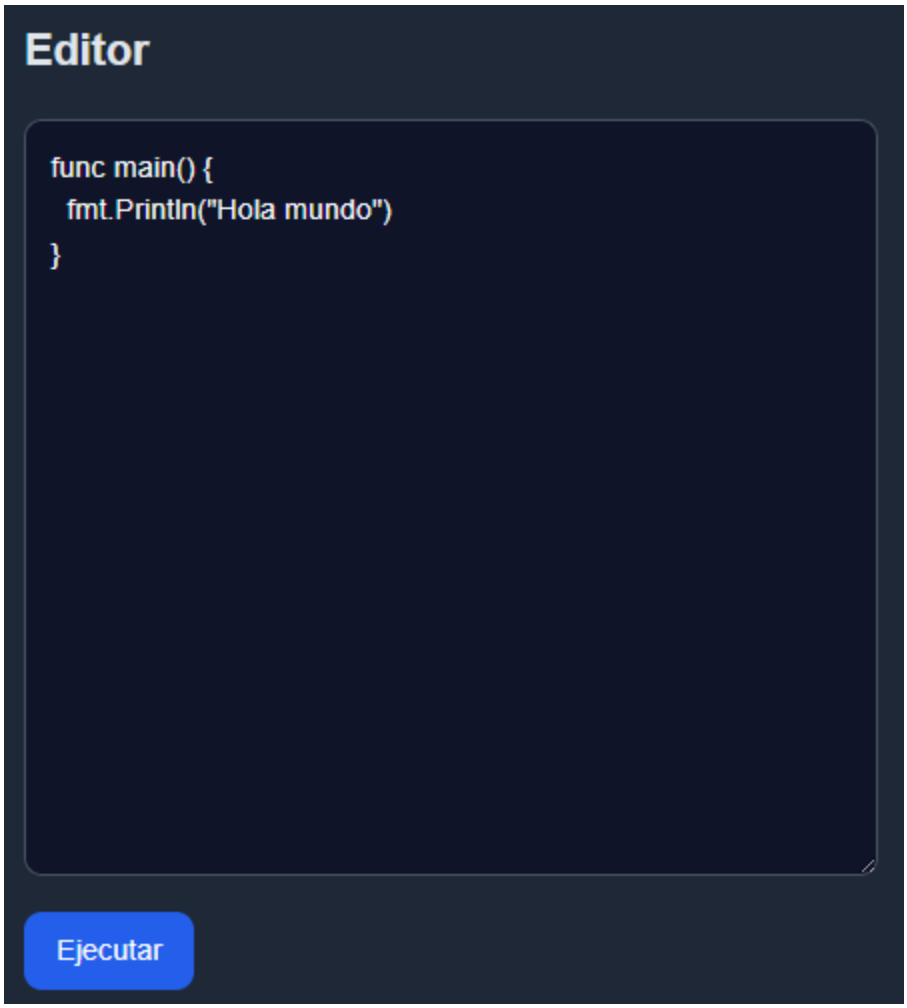
- consola
- tabla de errores
- tabla de símbolos
- representación DOT del AST

8. Funcionamiento interno del sistema

El flujo interno del sistema puede describirse en varias etapas consecutivas.

8.1 Recepción del código fuente

El usuario escribe un programa en el editor del frontend. Al ejecutar, el contenido se envía al backend.



8.2 Análisis léxico

La entrada es recorrida por el lexer para identificar tokens como identificadores, números, cadenas, operadores, palabras reservadas, signos de puntuación y delimitadores.

8.3 Análisis sintáctico

El parser valida que la secuencia de tokens cumpla la gramática del lenguaje. Si la entrada es válida, se construye el AST.

Tabla de símbolos

ID	Tipo símbolo	Tipo dato	Ámbito	Línea	Columna
main	Función	void	Global	1	6

8.4 Construcción del AST

Cada producción relevante de la gramática genera nodos de AST con atributos como:

- tipo de nodo
- valor
- línea
- columna
- hijos

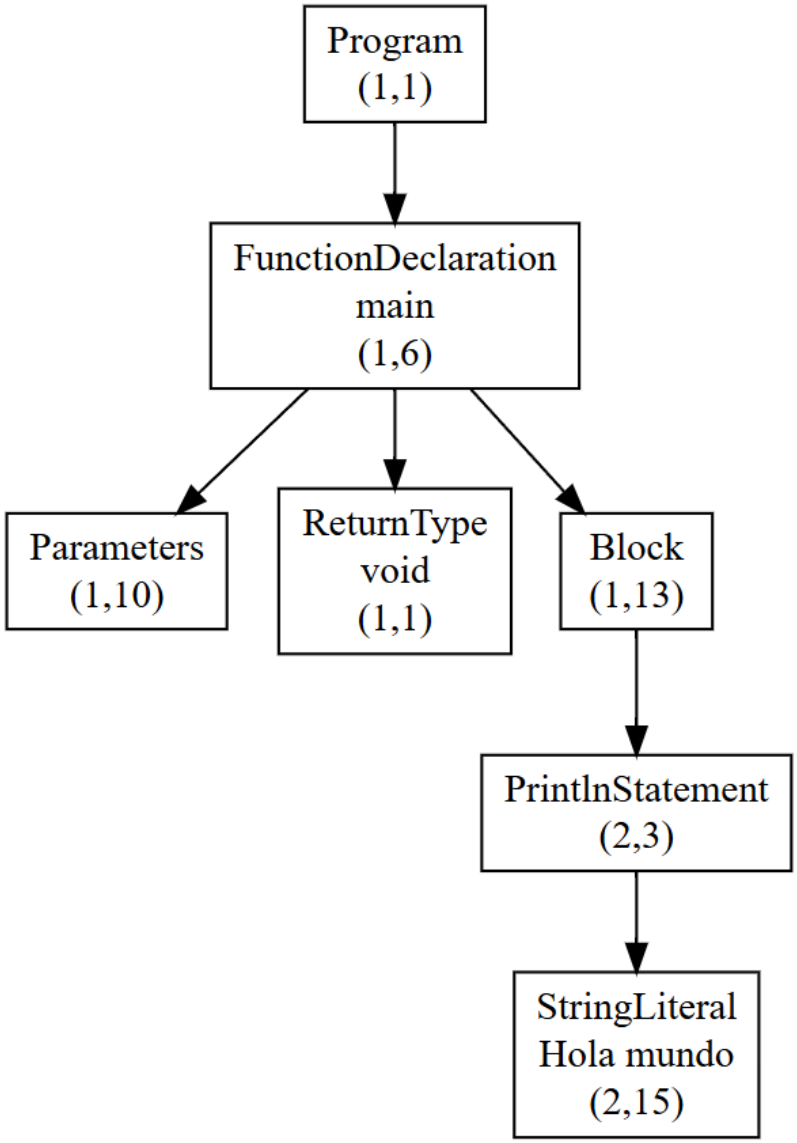
AST en formato DOT

```
digraph AST {
  node [shape=box];
  n0 [label="Program\n(1,1)"];
  n1 [label="FunctionDeclaration\nmain\n(1,6)"];
  n2 [label="Parameters\n(1,10)"];
  n1 -> n2;
  n3 [label="ReturnType\nvoid\n(1,1)"];
  n1 -> n3;
  n4 [label="Block\n(1,13)"];
  n5 [label="PrintlnStatement\n(2,3)"];
  n6 [label="StringLiteral\nHola mundo\n(2,15)"];
  n5 -> n6;
  n4 -> n5;
  n1 -> n4;
  n0 -> n1;
}
```

8.5 Análisis semántico e interpretación

El AST se recorre para:

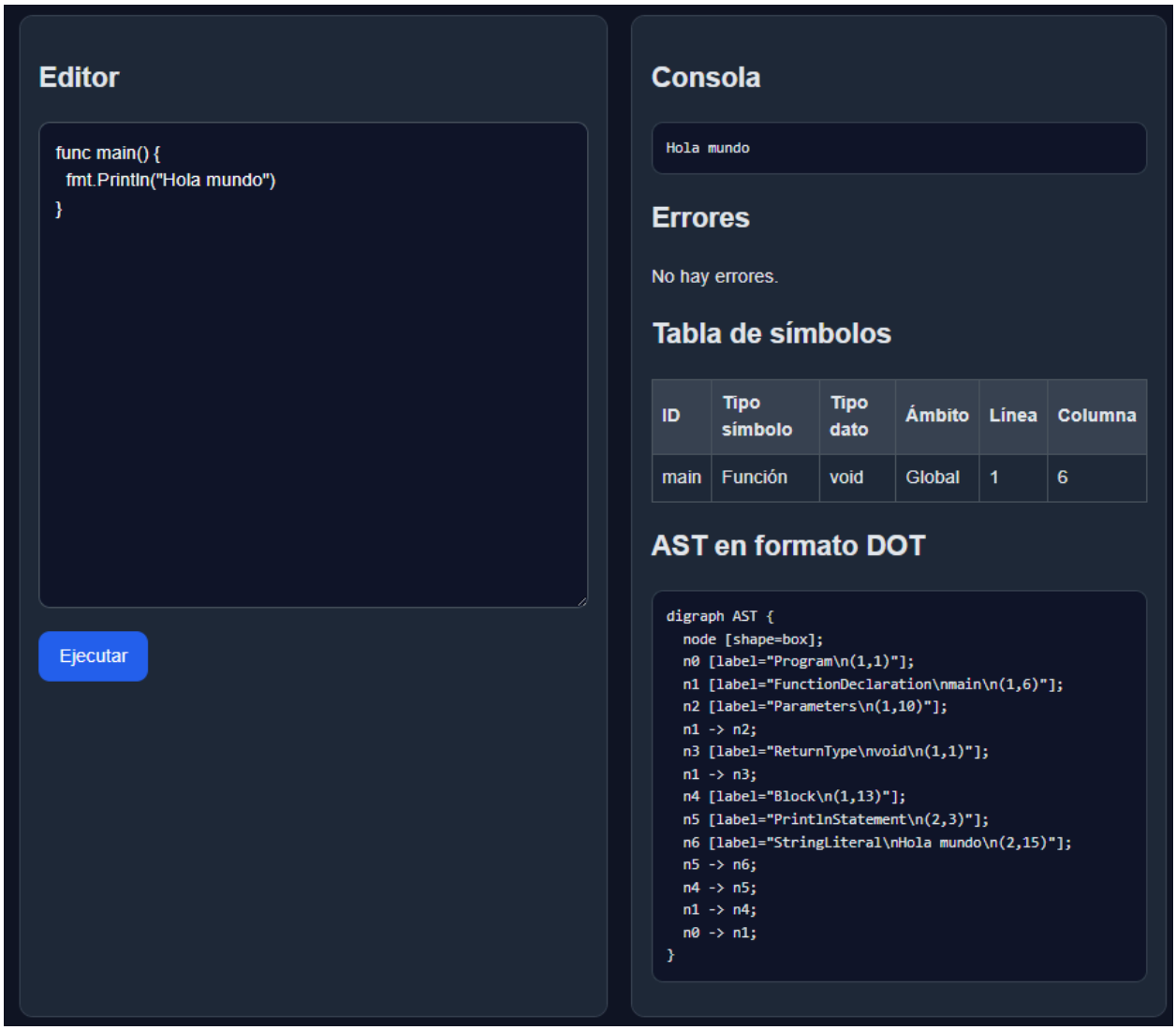
- declarar variables
- validar tipos
- ejecutar expresiones
- ejecutar sentencias
- aplicar funciones embebidas
- construir tabla de símbolos
- detectar errores



8.6 Generación de respuesta

El backend devuelve una respuesta estructurada que contiene:

- salida de consola
- errores
- tabla de símbolos
- AST en DOT



9. Análisis léxico

El análisis léxico se implementó en la gramática de Jison mediante reglas que reconocen los tokens del lenguaje. Entre los elementos conocidos se encuentran:

- palabras reservadas como func, if, else, for, switch, case, default, return, struct, range
- tipos primitivos como int, float64, string, bool, rune
- operadores aritméticos, relacionales y lógicos
- operadores de asignación como =, :=, +=, -=
- delimitadores y signos de agrupación
- identificadores
- literales enteros, flotantes, cadenas, booleanos y rune

```
%token FUNC VAR TYPE STRUCT FMT PRINTLN STRCONV STRINGS SLICES REFLECT TYPEOF TYPE_INT TYPE_FLOAT64 TYPE_STRING TYPE_BOOL TYPE_RUNE
%token IDENTIFIER STRING INT FLOAT BOOL RUNE DECLARE EOF
%token EQ NEQ GTE LTE AND OR NOT IF ELSE FOR INC DEC BREAK CONTINUE RETURN RANGE
%token SWITCH CASE DEFAULT
%token PLUS_ASSIGN MINUS_ASSIGN

%left OR
%left AND
%left EQ NEQ
%left '<' '>' GTE LTE
%left '+' '-'
%left '*' '/' '%'
%right NOT UMINUS

%start program

%lex
%options flex
```

```
%%
[ \t\r\n]+                                  /* ignorar espacios */
"//"[^\n]*                                  /* comentario una línea */

"func"                                      return 'FUNC';
"var"                                       return 'VAR';
"type"                                      return 'TYPE';
"struct"                                    return 'STRUCT';
"fmt"                                       return 'FMT';
"Println"                                   return 'PRINTLN';
"strconv"                                  return 'STRCONV';
"strings"                                   return 'STRINGS';
"slices"                                    return 'SLICES';
"reflect"                                   return 'REFLECT';
"TypeOf"                                    return 'TYPEOF';
"if"                                        return 'IF';
"else"                                      return 'ELSE';
"for"                                       return 'FOR';
"break"                                     return 'BREAK';
"continue"                                  return 'CONTINUE';
"return"                                    return 'RETURN';
"range"                                     return 'RANGE';
"switch"                                    return 'SWITCH';
"case"                                      return 'CASE';
"default"                                   return 'DEFAULT';

"int"                                       return 'TYPE_INT';
"float64"                                   return 'TYPE_FLOAT64';
"string"                                    return 'TYPE_STRING';
"bool"                                      return 'TYPE_BOOL';
"rune"                                      return 'TYPE_RUNE';

"true"|"false"                              return 'BOOL';

":="                                        return 'DECLARE';
"=="                                        return 'EQ';
"!="                                        return 'NEQ';
">="                                        return 'GTE';
"<="                                        return 'LTE';
"&&"                                        return 'AND';
"||"                                        return 'OR';
"++"                                        return 'INC';
```

```
"--"                                    return 'DEC';
"+="                                    return 'PLUS_ASSIGN';
"-="                                    return 'MINUS_ASSIGN';

"!"                                     return 'NOT';
":"                                     return ':';
"="                                     return '=';
","                                     return ',';
";"                                     return ';';
"."                                     return '.';
"("                                     return '(';
")"                                     return ')';
"{"                                     return '{';
"}"                                     return '}';
"["                                     return '[';
"]"                                     return ']';
"+"                                     return '+';
"-"                                     return '-';
"*"                                     return '*';
"/"                                     return '/';
"%"                                     return '%';
">"                                     return '>';
"<"                                     return '<';

[0-9]+\.[0-9]+                          return 'FLOAT';
[0-9]+                                  return 'INT';

\"([^\\"]|\\["\\nrt])*\"                return 'STRING';
\'([^\\']|\\[nrt\\'"\\])*\'             return 'RUNE';

[a-zA-Z_][a-zA-Z0-9_]*                  return 'IDENTIFIER';

```

Cuando se detecta un símbolo inválido, el sistema registra un error con información de línea y columna.

10. Análisis sintáctico

El análisis sintáctico se basa en una gramática diseñada en Jison. Esta gramática define la estructura válida del lenguaje y construye nodos del AST conforme se reducen las producciones.

Entre las construcciones soportadas se encuentran:

- declaraciones de variables
- declaraciones tipadas
- structs
- funciones con parámetros y retorno
- expresiones
- llamadas a funciones
- condicionales
- ciclos for
- switch
- arreglos y slices
- acceso por índice
- acceso a atributos
- literales de struct
- funciones embebidas

```
/*
  ============================================================
  STRUCTS
  ============================================================
  Soporta:
    type Persona struct { ... }
    struct Persona { ... }
*/

struct_decl
    : TYPE IDENTIFIER STRUCT '{' struct_field_items_opt '}'
        {
          $$ = createNode('StructDeclaration', $2, @2, $5);
        }
    | STRUCT IDENTIFIER '{' struct_field_items_opt '}'
        {
          $$ = createNode('StructDeclaration', $2, @2, $4);
        }
    ;

struct_field_items_opt
    : struct_field_items
        { $$ = $1; }
    |
        { $$ = []; }
    ;

struct_field_items
    : struct_field_items struct_field_item
        { $$ = $1.concat([$2]); }
    | struct_field_item
        { $$ = [$1]; }
    ;

struct_field_item
    : struct_field_decl struct_field_sep_opt
        { $$ = $1; }
    ;

struct_field_sep_opt
    : ';'
        { $$ = null; }
    |
```

```
312     ============================================================
313     FUNCIONES
314     ============================================================
315   */
316
317   function_decl
318       : FUNC IDENTIFIER '(' param_list_opt ')' return_type_opt block
319           {
320             $$ = createNode('FunctionDeclaration', $2, @2, [
321               createNode('Parameters', null, @4, $4),
322               $6,
323               $7
324             ]);
325           }
326       ;
327
328   param_list_opt
329       : param_list
330           { $$ = $1; }
331       |
332           { $$ = []; }
333       ;
334
335   param_list
336       : param_list ',' param_decl
337           { $$ = $1.concat([$3]); }
338       | param_decl
339           { $$ = [$1]; }
340       ;
341
342   param_decl
343       : IDENTIFIER callable_type
344           {
345             $$ = createNode('Parameter', null, @1, [
346               createNode('Identifier', $1, @1, []),
347               $2
348             ]);
349           }
350       ;
351
```

```
/*
  ============================================================
  PRINTLN
  ============================================================
*/

println_stmt
    : FMT '.' PRINTLN '(' expr_list_opt ')'
        { $$ = createNode('PrintlnStatement', null, @1, $5); }
    ;

/*
  ============================================================
  IF
  ============================================================
*/

if_stmt
    : IF expression block else_part_opt
        {
          var children = [$2, $3];
          if ($4) {
            children.push($4);
          }
          $$ = createNode('IfStatement', null, @1, children);
        }
    ;

else_part_opt
    : ELSE if_stmt
        { $$ = createNode('ElseBranch', null, @1, [$2]); }
    | ELSE block
        { $$ = createNode('ElseBranch', null, @1, [$2]); }
    |
        { $$ = null; }
    ;

/*
```

```
460     ============================================================
461     FOR
462     ============================================================
463     Soporta:
464       for condicion { ... }
465       for init ; cond ; update { ... }
466       for ; cond ; update { ... }
467       for i, valor := range numeros { ... }
468       for i := range numeros { ... }
469       for _, valor := range numeros { ... }
470       for i, _ := range numeros { ... }
471
472     IMPORTANTE:
473     Esta versión evita el conflicto de:
474       for contador < 5 { ... }
475
476     porque ya no usa un init opcional "codicioso" antes de decidir
477     si es for clásico o for por condición.
478   */
479
480   for_stmt
481       : FOR range_clause block
482           {
483             $$ = createNode('ForRangeStatement', null, @1, [
484               $2.binding,
485               $2.iterable,
486               $3
487             ]);
488           }
489       | FOR for_classic_header block
490           {
491             $$ = createNode('ForStatement', 'classic', @1, [
492               $2.init,
493               $2.condition,
494               $2.update,
495               $3
496             ]);
497           }
498       | FOR expression block
499           {
500             $$ = createNode('ForStatement', 'condition', @1, [$2, $3]);
```

```
633 /*
634   ============================================================
635   SWITCH
636   ============================================================
637 */
638
639 switch_stmt
640     : SWITCH switch_subject '{' case_clause_list_opt default_clause_opt '}'
641         {
642           var children = [$2];
643           if ($4) {
644             children = children.concat($4);
645           }
646           if ($5) {
647             children.push($5);
648           }
649           $$ = createNode('SwitchStatement', null, @1, children);
650         }
651     ;
652
653 /*
654   El sujeto del switch se separa del no-terminal "expression"
655   para evitar el conflicto con literales de struct.
656
657   Ejemplo del problema:
658       switch nota {
659           case 100:
660               ...
661       }
662
663   Si usamos "expression", el parser intenta leer "nota { ... }"
664   como si fuera un struct literal llamado "nota".
665 */
666 switch_subject
667     : switch_subject '.' IDENTIFIER
668         {
669           $$ = createNode('FieldAccess', $3, @2, [$1]);
670         }
671     | switch_subject '[' expression ']'
672         {
673           $$ = createNode('ArrayAccess', null, @2, [$1, $3]);
674         }
```

La gramática fue refinada progresivamente para soportar el estilo oficial del archivo de pruebas y evitar conflictos entre construcciones similares.

11. Construcción y uso del AST

El AST representa la estructura abstracta del programa. Cada nodo contiene:

- el tipo de construcción
- el valor textual cuando aplica
- la posición en línea y columna
- sus nodos hijos

Ejemplos de nodos usados en el proyecto:

- Program
- FunctionDeclaration
- Block
- VarDeclaration
- Assignment
- BinaryExpression
- IfStatement
- ForStatement
- ForRangeStatement
- SwitchStatement
- StructDeclaration
- StructLiteral
- FieldAccess
- ArrayAccess
- CallExpression
- QualifiedCallExpression

El AST es utilizado por el intérprete semántico para ejecutar el programa y también se convierte a formato DOT para visualización.

AST en formato DOT

```
digraph AST {
  node [shape=box];
  n0 [label="Program\n(1,1)"];
  n1 [label="FunctionDeclaration\nmain\n(1,6)"];
  n2 [label="Parameters\n(1,10)"];
  n1 -> n2;
  n3 [label="ReturnType\nvoid\n(1,1)"];
  n1 -> n3;
  n4 [label="Block\n(1,13)"];
  n5 [label="PrintlnStatement\n(2,3)"];
  n6 [label="StringLiteral\nHola mundo\n(2,15)"];
  n5 -> n6;
  n4 -> n5;
  n1 -> n4;
  n0 -> n1;
}
```

12. Análisis semántico e interpretación

El análisis semántico se realiza recorriendo el AST. Durante este proceso, el sistema:

- crea y administra scopes
- registra símbolos
- valida tipos
- ejecuta instrucciones
- evalúa expresiones
- controla el flujo del programa
- reporta errores semánticos

12.1 Manejo de scopes

Cada bloque, función, if, for, switch o bloque independiente puede generar un nuevo ámbito. Esto permite manejar correctamente la visibilidad de variables y parámetros.

12.2 Variables y tipos

El sistema soporta tipos primitivos, arrays, slices y structs. También valida asignaciones y operaciones entre tipos compatibles.

12.3 Control de flujo

Se soportan:

- if
- if else
- else if
- for por condición
- for clásico
- for range
- switch
- break
- continue
- return

12.4 Structs

Se soportan:

- declaración de structs
- instanciación
- acceso a campos
- asignación a campos
- structs anidados
- structs como parámetros y retorno

12.5 Arrays y slices

Se soportan:

- arrays fijos
- slices
- slices multidimensionales

- acceso y modificación por índice
- append
- len
- paso por referencia de slices

13. Tabla de símbolos

La tabla de símbolos se construye durante la ejecución del programa. Cada entrada registra información relevante sobre identificadores como variables, parámetros, funciones y structs.

Los campos principales de cada entrada son:

- identificador
- tipo de símbolo
- tipo de dato
- ámbito
- línea
- columna

La tabla de símbolos permite:

- verificar declaraciones
- detectar duplicados
- visualizar el estado estructural del programa
- apoyar el análisis semántico

AST en formato DOT

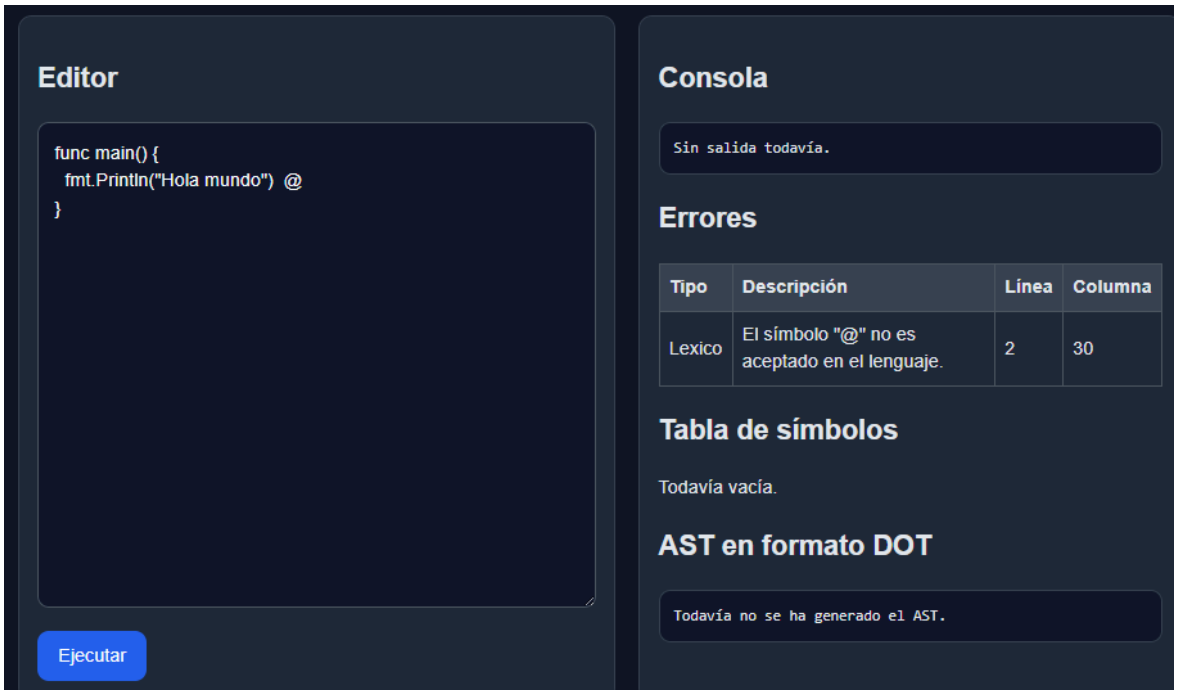
```
digraph AST {
  node [shape=box];
  n0 [label="Program\n(1,1)"];
  n1 [label="FunctionDeclaration\nmain\n(1,6)"];
  n2 [label="Parameters\n(1,10)"];
  n1 -> n2;
  n3 [label="ReturnType\nvoid\n(1,1)"];
  n1 -> n3;
  n4 [label="Block\n(1,13)"];
  n5 [label="PrintlnStatement\n(2,3)"];
  n6 [label="StringLiteral\nHola mundo\n(2,15)"];
  n5 -> n6;
  n4 -> n5;
  n1 -> n4;
  n0 -> n1;
}
```

14. Manejo de errores

El sistema maneja tres categorías principales de errores.

14.1 Errores léxicos

Se generan cuando el lexer encuentra símbolos no válidos para el lenguaje.



14.2 Errores sintácticos

Se producen cuando la secuencia de tokens no cumple con la gramática definida.

15. Funcionalidades implementadas

El sistema implementa las siguientes características principales:

- Declaración y uso de variables.

Tipos primitivos: int, float64, string, bool, rune.

- Operaciones aritméticas:

suma

resta

multiplicación

división

módulo

- Operaciones relacionales:

==

!=

>

<

>=

<=

Operaciones lógicas:

&&

| |

!

- Asignaciones simples y compuestas:

=

:=

+=

-=

- Estructuras de control:

if

else if

else

for

switch

- Control de flujo:

break

continue

return

- Funciones:

declaración

parámetros

retorno

llamadas

- Arreglos y slices:

arrays

slices

matrices

acceso por índice

modificación por índice

- Structs:

declaración

structs anidados

acceso a atributos

modificación de atributos

- Visualización de:

consola

errores

tabla de símbolos

AST

16. Funciones embebidas soportadas

El sistema incorpora varias funciones embebidas utilizadas en las pruebas del proyecto.

16.1 len

Obtiene la longitud de arrays o slices.

16.2 append

Agrega elementos a un slice.

16.3 strconv.Atoi

Convierte cadenas válidas a enteros.

16.4 strconv.ParseFloat

Convierte cadenas válidas a números de tipo float64.

16.5 strings.Join

Une elementos de un slice de strings usando un separador.

16.6 slices.Index

Busca un elemento dentro de un array o slice y retorna su posición.

16.7 reflect.TypeOf

Devuelve el tipo del valor evaluado, incluyendo soporte para .string().

17. Pruebas realizadas

Durante el desarrollo se realizaron pruebas unitarias y progresivas para validar cada parte del sistema. Entre las pruebas realizadas se incluyen:

- Pruebas de expresiones aritméticas y lógicas.
- Pruebas de comparaciones.
- Pruebas de variables y asignaciones.
- Pruebas de if, for y switch.
- Pruebas de break, continue y return.
- Pruebas de funciones con parámetros y retorno.
- Pruebas de arrays, slices y matrices.
- Pruebas de range.
- Pruebas de structs y structs anidados.
- Pruebas de arreglos y slices de structs.
- Pruebas de funciones embebidas oficiales.
- Pruebas con el archivo oficial proporcionado para el proyecto.

```
=== PRUEBA GENERAL DE GOSCRIPT ===
10 15.5 Compiladores true A
Entero: 15
Decimal: 13.5
Texto: Compiladores 1
Bandera: false
mezcla1: 6
mezcla2: 12.5
mezcla3: 70
mezcla4: Valor: 50
mezcla5: 2.5
mezcla6: 66
Atoi: 123
ParseFloat: 45.67
Tipo entero: int
Tipo decimal: float64
Tipo texto: string
Tipo bandera: bool
Slice numeros: [1, 2, 3, 4, 5]
Slice palabras: [OLC1, Proyecto, 2]
Join: OLC1 - Proyecto - 2
Index de 4: 3
Len numeros: 5
Slice actualizado: [1, 2, 3, 4, 5, 6, 7]
Suma del slice: 28
Impar: 1
Par: 2
Impar: 3
Par: 4
Impar: 5
Par: 6
Impar: 7
Slice modificado: [100, 12, 3, 4, 5, 6, 7]
Matriz: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
Elemento [1][2]: 6
Matriz modificada: [[99, 2, 3], [4, 5, 6], [7, 58, 9]]
Matriz extendida: [[99, 2, 3], [4, 5, 6], [7, 58, 9], [11, 12, 13]]
Direccion: Direccion{calle: Zona 1, numero: 100, principal: true}
Curso: Curso{nombre: Organizacion de Lenguajes, creditos: 4}
Estudiante: Estudiante{nombre: Luis, edad: 21, activo: true, direccion: Direccion{calle: Zona 1, numero: 100, principal: true},
curso: Curso{nombre: Organizacion de Lenguajes, creditos: 4}}
Nombre estudiante: Luis
Edad estudiante: 21
Calle: Zona 1
Curso actual: Organizacion de Lenguajes
Estudiante actualizado: Estudiante{nombre: Luis Manuel, edad: 22, activo: true, direccion: Direccion{calle: Zona 1, numero:
250, principal: true}, curso: Curso{nombre: Organizacion de Lenguajes, creditos: 5}}
Curso tras aumentar creditos: Curso{nombre: Organizacion de Lenguajes, creditos: 7}
Resultado suma: 40
Resultado promedio: 80.0
```

```
Es mayor de edad: true
Mensaje: Nombre: Luis Manuel, Edad: 22, Activo: true
El estudiante esta activo
Es mayor de edad
Condicion compleja verdadera
while-like contador: 0
while-like contador: 1
while-like contador: 2
while-like contador: 3
while-like contador: 4
for clasico: 0
for clasico: 1
for clasico: 3
for clasico: 4
for clasico: 5
for clasico: 6
range -> indice: 0 valor: 10
range -> indice: 1 valor: 20
range -> indice: 2 valor: 30
Estado de nota: Muy bien
La letra es A
op1: false
op2: true
op3: true
op4: true
op5: true
op6: true
calculo1: 20
calculo2: 7.5
calculo3: 20
calculo4: 1
calculo5: 2.5
acumulado: 12
acumulado2: 23.0
cadena final: Hola Mundo
bloque interno
999
segundo bloque
Acceso a entero externo: 15
=== FIN DE PRUEBA ===
```

18. Instalación y ejecución del proyecto

Para ejecutar el proyecto localmente, se deben seguir estos pasos:

18.1 Backend

Abrir una terminal en la carpeta del backend.

Instalar dependencias:

- npm install

Generar el parser:

- npm run generate-parser

Iniciar el backend:

- npm run dev

18.2 Frontend

Abrir una terminal en la carpeta del frontend.

Instalar dependencias:

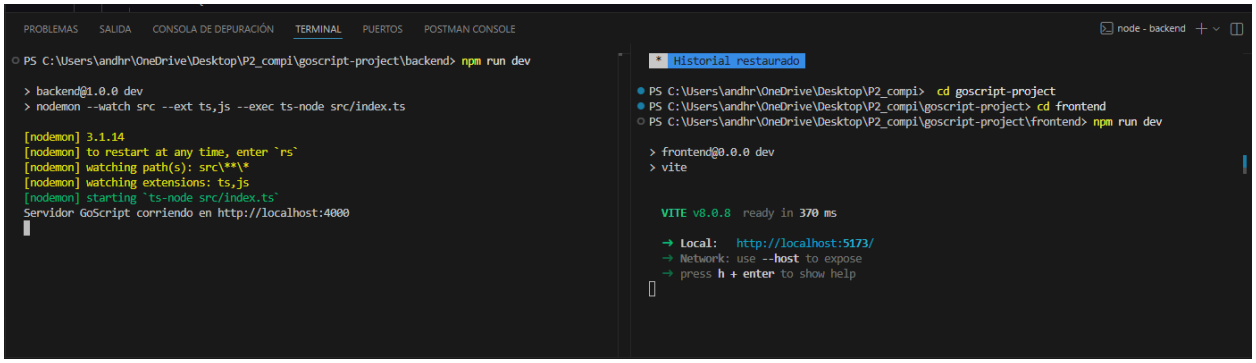
- npm install

Iniciar el frontend:

- npm run dev

18.3 Acceso

Abrir el navegador e ingresar a la dirección local indicada por el frontend.



.

19. Limitaciones del sistema

Aunque el sistema cumple con una parte amplia del lenguaje definido para el proyecto, presenta algunas limitaciones propias de una implementación académica.

Entre ellas se pueden mencionar:

- El sistema está orientado a un conjunto específico de construcciones del lenguaje GoScript.
- Algunos mensajes de error pueden mejorarse para ser más específicos.
- La salida DOT del AST puede optimizarse visualmente.

La interpretación depende de las reglas definidas en la gramática, por lo que extensiones futuras requieren modificar tanto el parser como el intérprete.

No se trata de un compilador completo de producción, sino de una herramienta académica orientada al análisis e interpretación.

20. Posibles mejoras futuras

Como trabajo futuro, el sistema podría ampliarse con:

- Mejora de mensajes de error.
- Mayor cobertura del lenguaje.
- Optimización del análisis semántico.
- Mejor visualización gráfica del AST.
- Exportación de reportes.
- Carga y guardado de archivos desde la interfaz.
- Soporte para más funciones embebidas o módulos estándar.
- Resaltado de sintaxis en el editor.

21. Conclusiones

El sistema GoScript IDE – Base del Proyecto logró integrar de forma funcional una interfaz gráfica con un backend de análisis e interpretación, permitiendo procesar programas del lenguaje GoScript de forma estructurada.

Entre los aportes más relevantes se encuentran la implementación de una gramática funcional en Jison, la construcción del AST, el manejo de la tabla de símbolos, la detección de errores y el soporte de varias construcciones del lenguaje, incluyendo funciones, structs, arrays, slices, ciclos, condicionales y funciones embebidas.

Desde el punto de vista técnico, el proyecto constituye una base sólida para comprender y aplicar conceptos fundamentales de teoría de compiladores, análisis de lenguajes y ejecución controlada de código fuente.